

Reviewer Pack — Minimal Rank of Quadratic Forms Bounds Disjointness Communic...

Entry 81e2b24727be · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 02:39:45 UTC

Minimal Rank of Quadratic Forms Bounds Disjointness Communication Complexity

Entry ID: 81e2b24727be **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 02:39:45 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Quadratic Forms) **Field B** (complexity object): Communication Complexity (Disjointness)

Statement:

[‘For any n -ary boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$, the minimal rank of the quadratic form matrix associated with the boolean function f lower bounds the randomized communication complexity of the disjointness problem for f .’, ‘Equivalently, if f is a boolean function representing the disjointness problem on n variables, then $\tau_{\text{QUAD}}(f) = \Omega(n \log n)$, where $\tau_{\text{QUAD}}(f)$ is the minimal rank of the quadratic form matrix associated with f .’]

Rationale (proposer’s reasoning):

[‘Quadratic forms provide a way to encode boolean functions in a more structured algebraic manner. The minimal rank of a quadratic form may capture the complexity of the underlying function and potentially relate it to communication complexity.’, ‘This conjecture aims to expose a connection between algebraic encoding and communication complexity, which could lead to new insights into the nature of disjointness problems.’]

Taxonomy category: COMM_DISJ (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e96708ffee4e485d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all n -ary boolean functions f with n in $\{5, \dots, 40\}$, $\tau_{\text{QUAD}}(f) / n \log n \geq 0.9$ AND no seed produces a communication complexity value $> 1.2\tau_{\text{QUAD}}(f)$. The conjecture is falsified if any seed produces either $\tau_{\text{QUAD}}(f) / n \log n < 0.7$ OR a communication complexity value $> 1.5\tau_{\text{QUAD}}(f)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal rank quadratic forms" AND "communication complexity disjointness" - " τ_{QUAD} boolean function" AND "randomized communication complexity" - "disjointness problem" AND τ_{QUAD} ("quadratic form matrix")

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def quadratic_form_matrix(f):
    n = len(f)
    M_f = [[0] * n for _ in range(n)]
    for x_i in range(2**n):
        for x_j in range(2**n):
            sum_product = 0
            for k in range(n):
                bit_i = (x_i >> k) & 1
                bit_j = (x_j >> k) & 1
                sum_product += f[(bit_i, bit_j)]
            M_f[x_i][x_j] = sum_product
    return M_f

def min_rank(matrix):
    n = len(matrix)
    rank = 0
    for i in range(n):
        if all(matrix[j][i] == 0 for j in range(i)):
            continue
        pivot_row = next(j for j in range(i, n) if matrix[j][i] != 0)
        matrix[i], matrix[pivot_row] = matrix[pivot_row], matrix[i]
        rank += 1
        for j in range(n):
            if i == j:
                continue
            factor = -matrix[j][i] / matrix[i][i]
            for k in range(i+1, n):
                matrix[j][k] += factor * matrix[i][k]
```

```

        for k in range(n):
            matrix[j][k] += factor * matrix[i][k]
    return rank

def communication_complexity(f, n):
    # Placeholder for the actual communication complexity algorithm
    # For this example, we assume a known upper bound
    return 2**n - 1

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    f = {tuple(sorted(random.sample(range(n), 2))): random.choice([0, 1]) for _ in range(2**n)}

    M_f = quadratic_form_matrix(f)
    tau_quad = min_rank(M_f)

    comm_complexity = communication_complexity(f, n)

    metric_value = Fraction(tau_quad) / (n * math.log(n))
    conjecture_holds = 0.9 <= metric_value and comm_complexity <= 1.2 * tau_quad
    counterexample = "" if conjecture_holds else "communication_complexity > 1.2*tau_quad"

    return {
        "metric_name": "minimal_rank",
        "metric_value": float(metric_value),
        "instances_tested": len(f),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = list(map(int, sys.argv[1:])) or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] and "communication_complexity > 1.2*tau_quad" in r["counterexample"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"communication_complexity > 1.2*tau_quad\" first={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot verify if the conjecture's support conditions are met. | next: Re-run the test with increased time limits to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11541
2	preregistration	ollama_remote	glm4:latest	0	0	6271
3	novelty	ollama_remote	glm4:latest	0	0	4627
4	novelty	ollama_remote	glm4:latest	0	0	5431
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14386
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14144
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10691
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9975
9	judge	ollama_remote	glm4:latest	0	0	8121

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 85186 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/81e2b24727be.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in [research/audit/81e2b24727be.jsonl](#) for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: [research/pvsnp_sandbox/test_*.py](#) (per-cycle)

Replay tarball: [research/replay/81e2b24727be.tar.gz](#) (if generated)